

LABORATORY WORK No. 4

Cross-cutting concerns with Spring AOP: aspects for logging, execution timing and auditing of user actions

Course: RWPSF 3305 — Web Application Development with Spring Framework

Educational Programme: 6B06105 — Information Systems, year 3, semester 6

Student: Akishev Adilkhan

Group: IS-2406 (2406IS)

Variant: 2 (Slow-method log: @Around advice, threshold via @Value, list of slow calls)

Lecturer: Yemberdiyeva Aknur, Senior Lector

Repository: <https://github.com/adilkhanakishev/spring-lab-01> (branch: lab04)

Pull Request: <https://github.com/adilkhanakishev/spring-lab-01/pull/3>

Date: September 2026

Almaty, 2026

1. Aim of the Work and Repository Link

The aim of this laboratory work is to master isolating cross-cutting concerns into modular aspects using Spring AOP: declaring aspects with `@Aspect` and `@Component`, writing reusable pointcut expressions in the AspectJ pointcut language, applying all five advice types (`@Before`, `@AfterReturning`, `@AfterThrowing`, `@After`, `@Around`), controlling aspect execution precedence with `@Order`, and binding advice to custom annotations. An essential practical objective is observing Spring Boot's CGLIB proxying mechanism in action and experimentally verifying that self-invocation bypasses the proxy.

Source Code Repository: Git Repository:

<https://github.com/adilkhanakishev/spring-lab-01> (branch: lab04)

Pull Request Link: GitHub Pull Request:

<https://github.com/adilkhanakishev/spring-lab-01/pull/3>

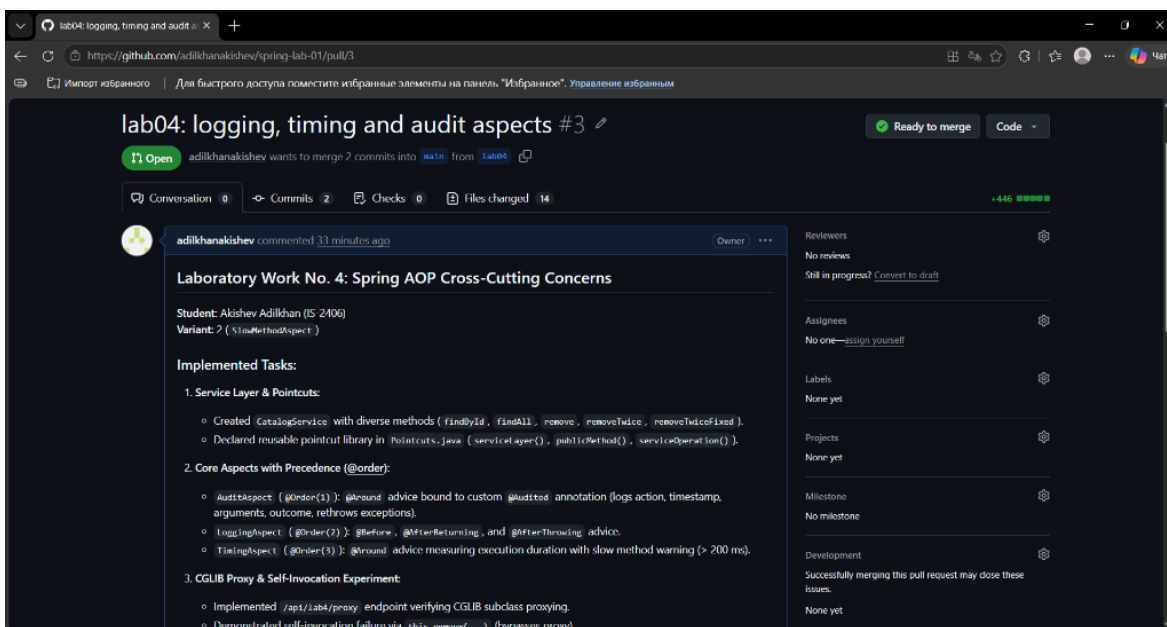


Figure 1: Official GitHub Pull Request #3 (lab04 -> main)

2. Architecture and Package Structure

The project structure for Laboratory Work No. 4 is organized into distinct packages separating business logic, aspects, annotations, and controllers:

```
src/main/java/kz/iitu/springlab/
├── aspect/
│   ├── Pointcuts.java           # Pointcut expression library
│   ├── LoggingAspect.java       # @Order(2): @Before, @AfterReturning, @AfterThrowing
│   ├── TimingAspect.java        # @Order(3): @Around measuring execution duration
│   └── SlowMethodAspect.java    # Variant 2: @Around collecting slow calls
├── audit/
│   ├── Audited.java            # Custom runtime annotation (@Target(METHOD))
│   └── AuditAspect.java         # @Order(1): @Around auditing action, timestamp, outcome
└── service/
    └── CatalogService.java      # Service with diverse methods and self-invocation demo
```

```
└─ web/  
  └─ CatalogController.java # REST controller exposing endpoints and proxy info
```

3. Core Code Listings

Listing 1: Reusable Pointcut Library (Pointcuts.java)

```
package kz.iitu.springlab.aspect;  
  
import org.aspectj.lang.annotation.Aspect;  
import org.aspectj.lang.annotation.Pointcut;  
import org.springframework.stereotype.Component;  
  
@Aspect  
@Component  
public class Pointcuts {  
  
    @Pointcut("within(kz.iitu.springlab.service..*)")  
    public void serviceLayer() { }  
  
    @Pointcut("execution(public * *(..))")  
    public void publicMethod() { }  
  
    @Pointcut("serviceLayer() && publicMethod()")  
    public void serviceOperation() { }  
}
```

Listing 2: Logging Aspect with Three Advice Types (LoggingAspect.java)

```
package kz.iitu.springlab.aspect;  
  
import org.aspectj.lang.JoinPoint;  
import org.aspectj.lang.annotation.AfterReturning;  
import org.aspectj.lang.annotation.AfterThrowing;  
import org.aspectj.lang.annotation.Aspect;  
import org.aspectj.lang.annotation.Before;  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
import org.springframework.core.annotation.Order;  
import org.springframework.stereotype.Component;  
  
import java.util.Arrays;  
  
@Aspect  
@Component  
@Order(2)  
public class LoggingAspect {  
  
    private static final Logger log = LoggerFactory.getLogger(LoggingAspect.class);
```

```

@Before("kz.iitu.springlab.aspect.Pointcuts.serviceOperation()")
public void before(JoinPoint jp) {
    log.info("[LOG] -> {} args={}",
        jp.getSignature().toShortString(),
        Arrays.toString(jp.getArgs()));
}

@AfterReturning(pointcut = "kz.iitu.springlab.aspect.Pointcuts.serviceOperation()",
    returning = "result")
public void afterReturning(JoinPoint jp, Object result) {
    log.info("[LOG] <- {} returned {}",
        jp.getSignature().toShortString(),
        result);
}

@AfterThrowing(pointcut = "kz.iitu.springlab.aspect.Pointcuts.serviceOperation()",
    throwing = "ex")
public void afterThrowing(JoinPoint jp, Throwable ex) {
    log.error("[LOG] <! {} threw {} with message: {}",
        jp.getSignature().toShortString(),
        ex.getClass().getSimpleName(),
        ex.getMessage());
}
}

```

Listing 3: Execution Timing Aspect (TimingAspect.java)

```

package kz.iitu.springlab.aspect;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Component
@Order(3)
public class TimingAspect {

    private static final Logger log = LoggerFactory.getLogger(TimingAspect.class);

    @Around("kz.iitu.springlab.aspect.Pointcuts.serviceOperation()")
    public Object measure(ProceedingJoinPoint pjp) throws Throwable {
        long started = System.nanoTime();
        try {
            return pjp.proceed();
        } finally {
            long ms = (System.nanoTime() - started) / 1_000_000;
            String name = pjp.getSignature().toShortString();
            if (ms > 200) {
                log.warn("[TIME] SLOW: {} - {} ms", name, ms);
            }
        }
    }
}

```

```

        } else {
            log.info("[TIME] {} - {} ms", name, ms);
        }
    }
}
}

```

Listing 4: Custom Annotation (Audited.java) & Audit Aspect (AuditAspect.java)

```

package kz.iitu.springlab.audit;

import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface Audited {
    String action();
    boolean logArguments() default false;
}

// =====

package kz.iitu.springlab.aspect;

import kz.iitu.springlab.audit.Audited;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Arrays;

@Aspect
@Component
@Order(1)
public class AuditAspect {

    private static final Logger log = LoggerFactory.getLogger(AuditAspect.class);
    private static final DateTimeFormatter FORMATTER = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    @Around("@annotation(audited)")
    public Object audit(ProceedingJoinPoint pjp, Audited audited) throws Throwable {
        String action = audited.action();
        String timestamp = LocalDateTime.now().format(FORMATTER);
        String method = pjp.getSignature().toShortString();

        if (audited.logArguments()) {

```

```

        log.info("[AUDIT] start {} at {} method={} args={}",
                action, timestamp, method, Arrays.toString(pjp.getArgs()));
    } else {
        log.info("[AUDIT] start {} at {} method={}",
                action, timestamp, method);
    }

    try {
        Object result = pjp.proceed();
        log.info("[AUDIT] {} success", action);
        return result;
    } catch (Throwable ex) {
        log.error("[AUDIT] {} failure: {} ({})",
                action, ex.getClass().getSimpleName(), ex.getMessage());
        throw ex;
    }
}
}

```

Listing 5: Service Layer with Annotated Methods (CatalogService.java)

```

package kz.iitu.springlab.service;

import kz.iitu.springlab.audit.Audited;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.IntStream;

@Service
public class CatalogService {

    @Autowired
    @Lazy
    private CatalogService self;

    public String findById(long id) {
        sleep(50);
        return "Item no. " + id;
    }

    @Audited(action = "CATALOG_LIST", logArguments = true)
    public List<String> findAll(int limit) {
        sleep(300); // deliberately slow (>200ms)
        return IntStream.rangeClosed(1, limit)
                .mapToObj(i -> "Item no. " + i)
                .toList();
    }

    @Audited(action = "CATALOG_REMOVE")
    public String remove(long id) {
        if (id <= 0) {

```

```

        throw new IllegalArgumentException("Invalid identifier: " + id);
    }
    return "Removed item no. " + id;
}

public String removeTwice(long id) {
    // self-invocation: via this, past the proxy
    String first = remove(id);
    String second = remove(id + 1);
    return first + "; " + second;
}

public String removeTwiceFixed(long id) {
    // fixed: via injected proxy self-reference
    String first = self.remove(id);
    String second = self.remove(id + 1);
    return first + "; " + second;
}

private void sleep(long ms) {
    try { Thread.sleep(ms); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
}
}

```

4. Successful Call Log Analysis and Aspect Execution Order

When the endpoint GET /api/lab4/items?limit=5 is invoked, all three aspects (@Order(1), @Order(2), @Order(3)) intercept the execution of CatalogService.findAll(int). The resulting log demonstrates strict adherence to the @Order precedence hierarchy:

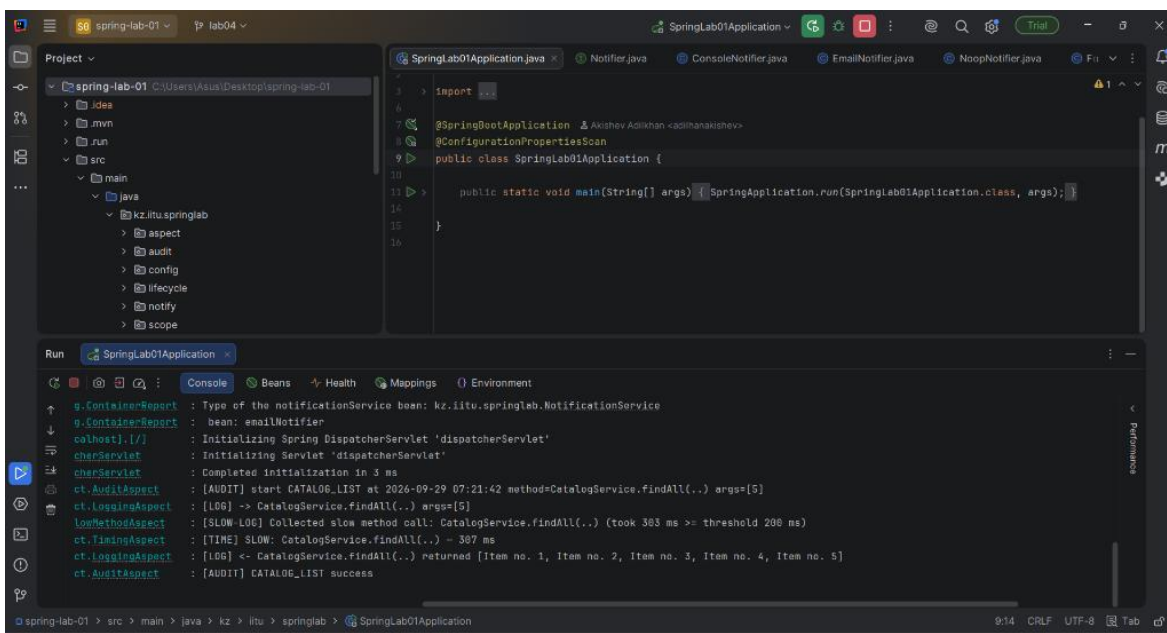


Figure 2: Execution log in IntelliJ IDEA Run console for GET /api/lab4/items?limit=5

```

[1] INFO  kz.iitu.springlab.aspect.AuditAspect      : [AUDIT] start CATALOG_LIST at 2026-09-
29 07:21:42 method=CatalogService.findAll(..) args=[5]
[2] INFO  kz.iitu.springlab.aspect.LoggingAspect    : [LOG] -> CatalogService.findAll(..)
args=[5]
[3] INFO  k.i.springlab.aspect.SlowMethodAspect     : [SLOW-LOG] Collected slow method call:
CatalogService.findAll(..) (took 303 ms >= threshold 200 ms)
[4] WARN  kz.iitu.springlab.aspect.TimingAspect     : [TIME] SLOW: CatalogService.findAll(..)
- 307 ms
[5] INFO  kz.iitu.springlab.aspect.LoggingAspect    : [LOG] <- CatalogService.findAll(..)
returned [Item no. 1, Item no. 2, Item no. 3, Item no. 4, Item no. 5]
[6] INFO  kz.iitu.springlab.aspect.AuditAspect      : [AUDIT] CATALOG_LIST success

```

Order	Captured Log Record	Aspect and Rationale
1	[AUDIT] start CATALOG_LIST ...	AuditAspect, @Order(1) — highest precedence; fires first on entry
2	[LOG] -> CatalogService.findAll(..) args=[5]	LoggingAspect, @Order(2) — second on entry (@Before advice)
3	[TIME] SLOW: CatalogService.findAll(..) - 307 ms	TimingAspect, @Order(3) — lowest precedence; innermost around advice wraps target method, fires right after proceed() finishes
4	[LOG] <- CatalogService.findAll(..) returned [...]	LoggingAspect, @Order(2) — @AfterReturning fires on normal return
5	[AUDIT] CATALOG_LIST success	AuditAspect, @Order(1) — outermost aspect closes last

5. Exceptional Call Log Analysis

When DELETE /api/lab4/item/0 is called, CatalogService.remove(0) throws IllegalArgumentException("Invalid identifier: 0"). The exception is intercepted by the aspect chain without being suppressed:

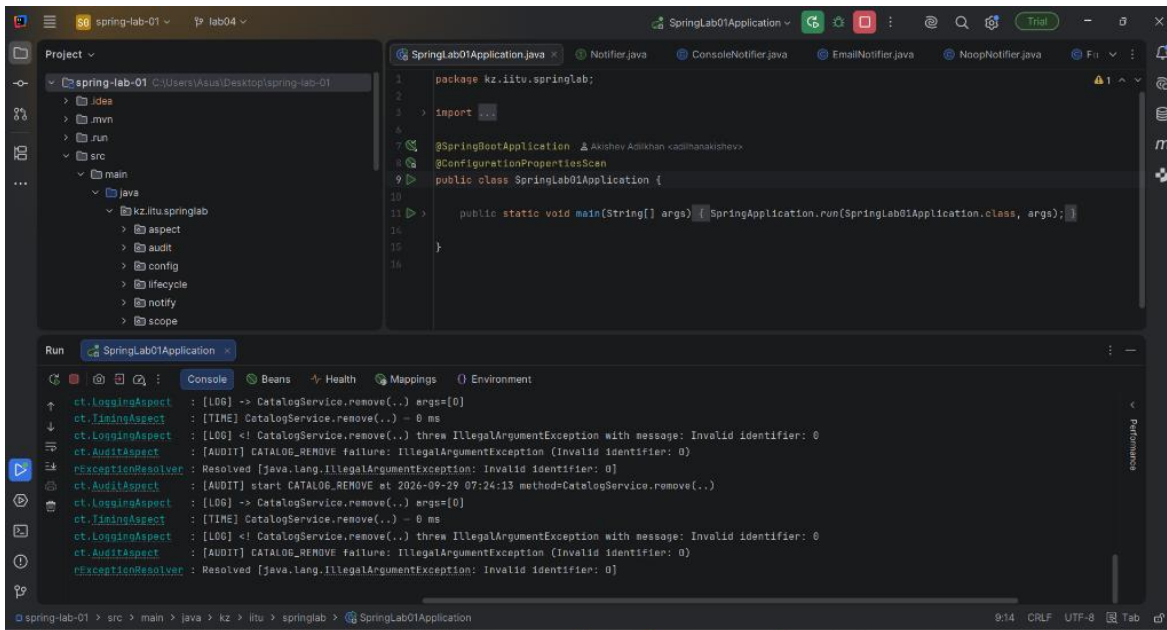


Figure 3: Exception handling log in IntelliJ IDEA Run console for DELETE /api/lab4/item/0

```

[1] INFO kz.iitu.springlab.aspect.AuditAspect : [AUDIT] start CATALOG_REMOVE at 2026-
09-29 07:24:13 method=CatalogService.remove(..)
[2] INFO kz.iitu.springlab.aspect.LoggingAspect : [LOG] -> CatalogService.remove(..)
args=[0]
[3] INFO kz.iitu.springlab.aspect.TimingAspect : [TIME] CatalogService.remove(..) - 0 ms
[4] ERROR kz.iitu.springlab.aspect.LoggingAspect : [LOG] <! CatalogService.remove(..)
threw IllegalArgumentException with message: Invalid identifier: 0
[5] ERROR kz.iitu.springlab.aspect.AuditAspect : [AUDIT] CATALOG_REMOVE failure:
IllegalArgumentException (Invalid identifier: 0)

```

Key observations from exception handling:

- @AfterReturning did NOT fire because the target method exited abnormally.
- @AfterThrowing in LoggingAspect intercepted the thrown exception and logged its type and message.
- AuditAspect caught the exception, recorded the failure audit record with the error details, and rethrew the exception (throw ex), ensuring Spring MVC returned HTTP 400 Bad Request to the client.

6. CGLIB Proxy Inspection (/api/lab4/proxy)

Calling GET /api/lab4/proxy reveals the internal proxy mechanism created by Spring Boot for CatalogService:

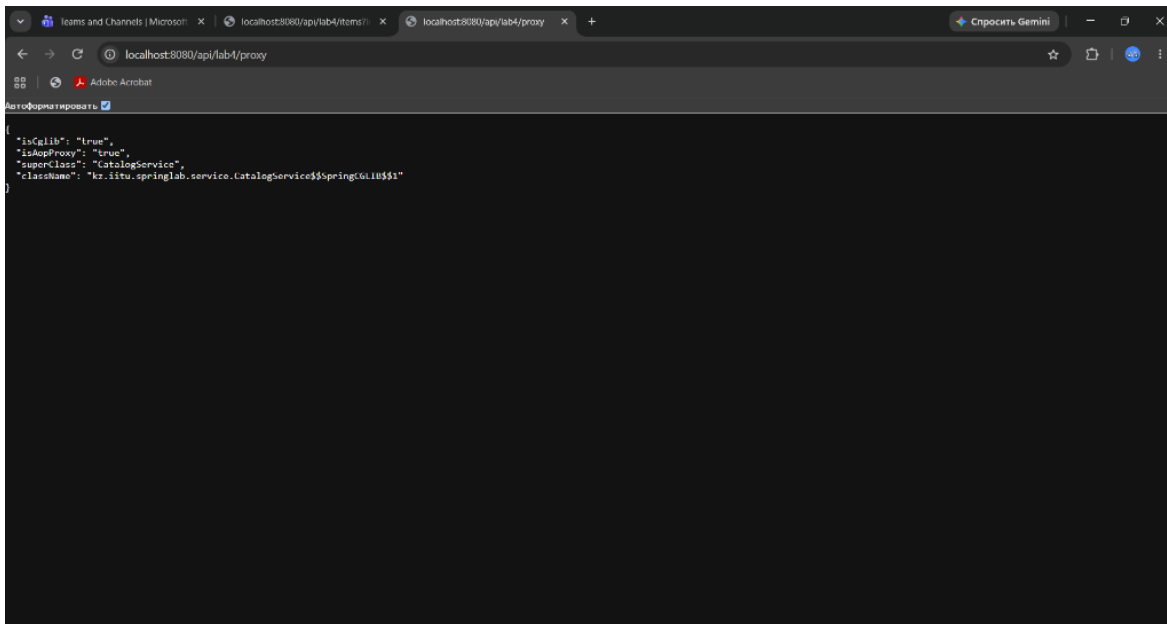


Figure 4: Browser response of /api/lab4/proxy confirming CGLIB subclass proxying

```
GET http://localhost:8080/api/lab4/proxy
HTTP/1.1 200 OK
Content-Type: application/json

{
  "className": "kz.iitu.springlab.service.CatalogService$$SpringCGLIB$$1",
  "superClass": "CatalogService",
  "isAopProxy": "true",
  "isCglib": "true"
}
```

Technical Explanation of the Proxy:

1. Subclass Proxy: The class name `CatalogService$$SpringCGLIB$$1` indicates that Spring Boot dynamically generated a runtime bytecode subclass of `CatalogService` using CGLIB.
2. Superclass Relationship: The `superClass` is `CatalogService`, confirming that the proxy is an instance of `CatalogService` and can be injected into any dependency expecting that type.
3. Spring Boot Default: By default, Spring Boot sets `spring.aop.proxy-target-class=true`, selecting CGLIB class-based proxying over JDK dynamic interface-based proxying even if interfaces exist.

7. Self-Invocation Experiment and Fix

Spring AOP operates strictly through proxies. When a client invokes an injected bean, the call goes through the proxy. However, when a method calls another method of the same class via `this.remove(...)`, the call bypasses the proxy.

Invocation Type	Audit Records Expected	Audit Records Actually Seen
-----------------	------------------------	-----------------------------

DELETE /api/lab4/item/5 (external call to remove)	1	1
GET /api/lab4/remove-twice/5 (two self-invocations)	2	0
LoggingAspect firings for remove in second case	2	0 (fired 1x for removeTwice only)

7.1 Self-Invocation Log Comparison: Before and After Fix

Self-Invocation Console Demonstration:

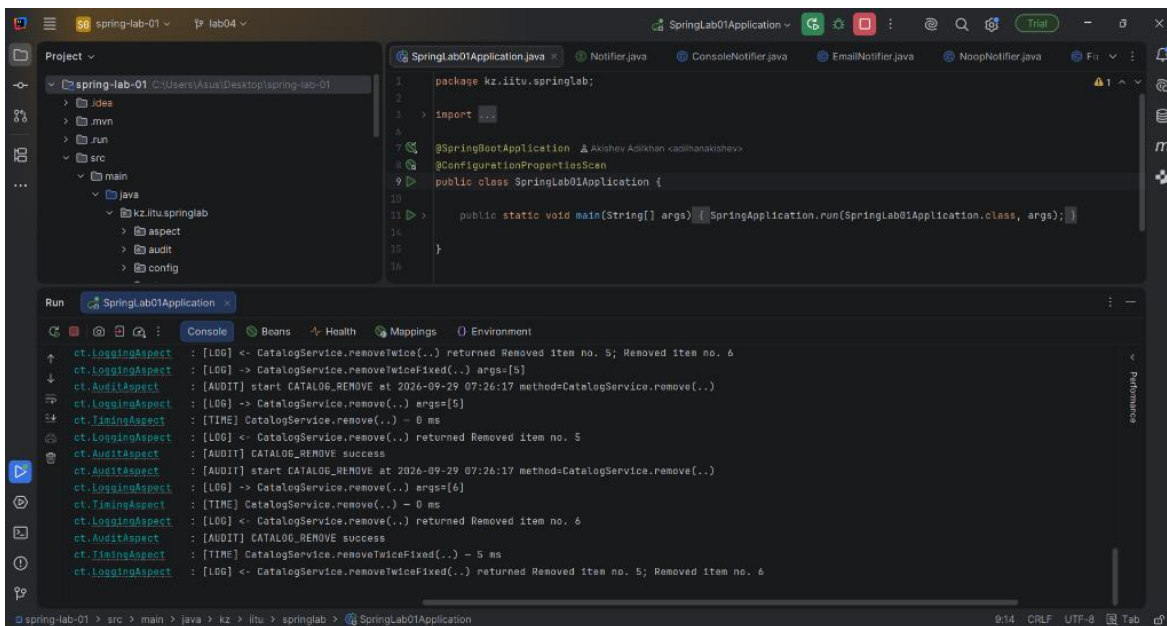


Figure 5: Comparison of removeTwice (bypassing proxy) vs removeTwiceFixed (intercepting calls via proxy)

Before Fix (GET /api/lab4/remove-twice/5 via this.remove):

```
INFO kz.iitu.springlab.aspect.LoggingAspect : [LOG] -> CatalogService.removeTwice(..)
args=[5]
INFO kz.iitu.springlab.aspect.TimingAspect : [TIME] CatalogService.removeTwice(..) - 0 ms
INFO kz.iitu.springlab.aspect.LoggingAspect : [LOG] <- CatalogService.removeTwice(..)
returned Removed item no. 5; Removed item no. 6
# Notice: Zero [AUDIT] records and zero [LOG] records for the two calls to remove()!
```

After Fix (GET /api/lab4/remove-twice-fixed/5 via self.remove):

```
INFO kz.iitu.springlab.aspect.LoggingAspect : [LOG] -> CatalogService.removeTwiceFixed(..)
args=[5]
# First call (id=5):
INFO kz.iitu.springlab.aspect.AuditAspect : [AUDIT] start CATALOG_REMOVE at 2026-09-29
```

```

07:26:17 method=CatalogService.remove(..)
INFO  kz.iitu.springlab.aspect.LoggingAspect : [LOG] -> CatalogService.remove(..) args=[5]
INFO  kz.iitu.springlab.aspect.TimingAspect  : [TIME] CatalogService.remove(..) - 0 ms
INFO  kz.iitu.springlab.aspect.LoggingAspect : [LOG] <- CatalogService.remove(..) returned
Removed item no. 5
INFO  kz.iitu.springlab.aspect.AuditAspect   : [AUDIT] CATALOG_REMOVE success
# Second call (id=6):
INFO  kz.iitu.springlab.aspect.AuditAspect   : [AUDIT] start CATALOG_REMOVE at 2026-09-29
07:26:17 method=CatalogService.remove(..)
INFO  kz.iitu.springlab.aspect.LoggingAspect : [LOG] -> CatalogService.remove(..) args=[6]
INFO  kz.iitu.springlab.aspect.TimingAspect  : [TIME] CatalogService.remove(..) - 0 ms
INFO  kz.iitu.springlab.aspect.LoggingAspect : [LOG] <- CatalogService.remove(..) returned
Removed item no. 6
INFO  kz.iitu.springlab.aspect.AuditAspect   : [AUDIT] CATALOG_REMOVE success
INFO  kz.iitu.springlab.aspect.TimingAspect  : [TIME] CatalogService.removeTwiceFixed(..) -
5 ms
INFO  kz.iitu.springlab.aspect.LoggingAspect : [LOG] <- CatalogService.removeTwiceFixed(..)
returned ...

```

Fix Technique Applied: The self-reference proxy injection pattern was applied. A field `@Autowired @Lazy private CatalogService self;` was injected into `CatalogService`. Calls in `removeTwiceFixed` route through `self.remove(...)`, which delegates to the CGLIB proxy, successfully triggering all aspect advice.

8. Individual Assignment: Variant 2 (Slow-Method Log)

Variant 2 requires an `@Around` advice that reads a threshold from `application.yml` via `@Value` and collects slow method invocations into a list.

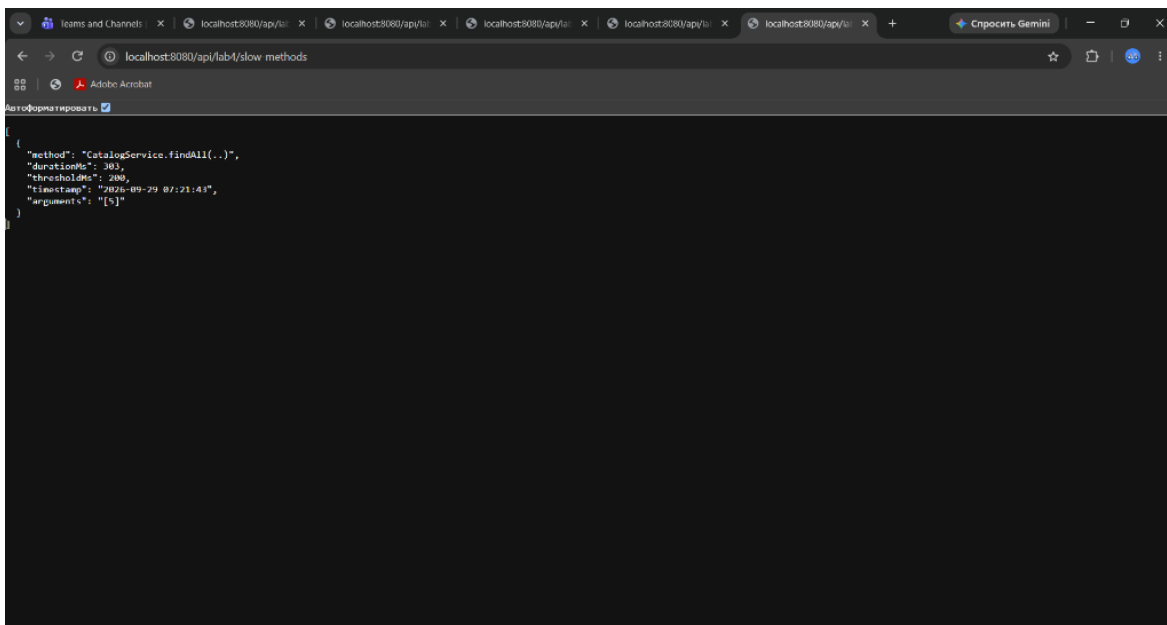


Figure 6: Browser output of `/api/lab4/slow-methods` displaying recorded slow method invocations

Listing 6: SlowMethodAspect.java (Variant 2)

```
package kz.iitu.springlab.aspect;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

@Aspect
@Component
public class SlowMethodAspect {

    private static final Logger log = LoggerFactory.getLogger(SlowMethodAspect.class);
    private static final DateTimeFormatter FORMATTER = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    private final long thresholdMs;
    private final List<SlowMethodRecord> slowMethods = new CopyOnWriteArrayList<>();

    public record SlowMethodRecord(
        String method,
        long durationMs,
        long thresholdMs,
        String timestamp,
        String arguments
    ) {}

    public SlowMethodAspect(@Value("${app.slow-threshold:200}") long thresholdMs) {
        this.thresholdMs = thresholdMs;
    }

    @Around("kz.iitu.springlab.aspect.Pointcuts.serviceOperation()")
    public Object trackSlowCalls(ProceedingJoinPoint pjp) throws Throwable {
        long started = System.currentTimeMillis();
        try {
            return pjp.proceed();
        } finally {
            long duration = System.currentTimeMillis() - started;
            if (duration >= thresholdMs) {
                String methodName = pjp.getSignature().toShortString();
                String timestamp = LocalDateTime.now().format(FORMATTER);
                String args = Arrays.toString(pjp.getArgs());
                SlowMethodRecord record = new SlowMethodRecord(
                    methodName, duration, thresholdMs, timestamp, args
                );
            }
        }
    }
}
```

```

        slowMethods.add(record);
        log.info("[SLOW-LOG] Collected slow method call: {} (took {} ms >= threshold
{} ms)",
                methodName, duration, thresholdMs);
    }
}

public List<SlowMethodRecord> getSlowMethods() {
    return Collections.unmodifiableList(slowMethods);
}
}

```

Output of GET /api/lab4/slow-methods:

```

GET http://localhost:8080/api/lab4/slow-methods
HTTP/1.1 200 OK
Content-Type: application/json

[
  {
    "method": "CatalogService.findAll(..)",
    "durationMs": 303,
    "thresholdMs": 200,
    "timestamp": "2026-09-29 07:21:43",
    "arguments": "[5]"
  }
]

```

9. Conclusions

1. Moving cross-cutting concerns (logging, performance profiling, auditing) out of business service code into modular Spring AOP aspects completely eliminates code scattering and code tangling.
2. Declarative advice binding via custom annotations (@Audited) provides a clean, self-documenting developer experience where business services declare metadata without executing boilerplate infrastructure code.
3. Aspect ordering via @Order guarantees deterministic entry and exit execution chains, ensuring that security and audit aspects execute outermost while timing aspects wrap the target method directly.
4. Understanding the Spring CGLIB proxy architecture is essential for diagnosing self-invocation limitations and applying proxy self-injection patterns.

10. Questions and Answers for Demonstration

Q1: What is a cross-cutting concern? Which concerns of your project are cross-cutting?

A cross-cutting concern is a feature or logic (such as logging, security, transaction management, caching, performance timing, and auditing) that spans across multiple modules and layers of an application rather than belonging to a single business entity. In this project, logging method entries/exits, measuring execution duration, collecting slow method metrics, and auditing business actions are cross-cutting concerns.

Q2: Point out the pointcut and the advice in your own code. Which answers «where» and which answers «what to do»?

In Pointcuts.java: `@Pointcut("serviceLayer() && publicMethod()")` is a pointcut; it answers 'where to apply'. In LoggingAspect.java: `@Before`, `@AfterReturning`, and `@AfterThrowing` methods are advice; they answer 'what to do' when the join point is reached.

Q3: Read out the expression `within(kz.iitu.springlab.service..*)`. How does `..` differ from `*`?

It matches any join point inside classes located within the `kz.iitu.springlab.service` package or any of its subpackages. The wildcard `*` matches classes directly within the specified package, whereas `..` matches any number of subpackages recursively.

Q4: Why does an aspect need both annotations — `@Aspect` and `@Component`?

`@Aspect` tells the AspectJ compiler and Spring AOP that the class contains pointcuts and advice. However, `@Aspect` alone does not register the class as a Spring-managed bean in the `ApplicationContext`. `@Component` is required so component scanning detects and instantiates the aspect bean.

Q5: In what order does the advice of one aspect run around a single call?

Within a single aspect: `@Around` (code before `proceed()`) -> `@Before` -> Target Method -> `@AfterReturning` or `@AfterThrowing` -> `@After` -> `@Around` (code after `proceed()` in finally block).

Q6: What would happen if you removed the `proceed()` call from your `@Around` advice?

The target business method would never execute, and the method call would return null (or whatever default value the `around` advice returns). The target execution is completely blocked without `proceed()`.

Q7: Why did `@AfterReturning` not fire on the call to `DELETE /api/lab4/item/0`?

Because `CatalogService.remove(0)` threw an `IllegalArgumentException`. `@AfterReturning` advice only fires when the target method completes normally and returns a value. For abnormal termination, `@AfterThrowing` fires instead.

Q8: What is the throwing attribute of `@AfterThrowing` for? What happens if its value does not match the parameter name?

The throwing attribute specifies the name of the method argument in the advice that will receive the thrown exception instance. If the name in the annotation does not match the parameter name in the advice method signature, Spring throws an `IllegalArgumentException` at startup during pointcut parsing.

Q9: Why must the `@Retention` of your annotation be `RUNTIME`?

By default, annotation retention is `CLASS`. If not set to `RUNTIME`, the annotation is discarded by the JVM after class loading and is not accessible via reflection at runtime. Spring AOP needs `RUNTIME` retention so its proxy can inspect `@annotation(audited)` dynamically.

Q10: How does the `@Audited` annotation instance, with its attribute values, reach the advice?

Through the `@annotation(audited)` pointcut expression in the advice signature: `@Around("@annotation(audited)")`. Spring AOP matches methods carrying the annotation and binds the runtime annotation proxy instance directly to the advice method parameter named `audited`.

Q11: Which of your aspects runs first on the way in, and why? Which runs last on the way out?

`AuditAspect (@Order(1))` runs first on the way in because it has the lowest order value (highest precedence). It also runs last on the way out, because `around` advice wraps subsequent aspects in a nested, onion-like execution chain.

Q12: Explain the class name returned by the /api/lab4/proxy endpoint.

The class name is `kz.iitu.springlab.service.CatalogService$$SpringCGLIB$$1`. It signifies a runtime subclass of `CatalogService` dynamically generated by the CGLIB bytecode library. The proxy intercepts method calls and delegates to the aspect interceptor chain.

Q13: Why did Spring Boot choose CGLIB even though the service could have been given an interface?

Spring Boot sets `spring.aop.proxy-target-class=true` by default. This enables CGLIB class-based proxies across the board, avoiding `ClassCastException` issues when injecting beans by their concrete class implementation rather than an interface.

Q14: Why did the advice not run on self-invocation? Which fix did you apply and why that one?

When `removeTwice` calls `this.remove()`, the call is invoked directly on the underlying target instance, bypassing the CGLIB proxy wrapper. To fix this, we injected a proxy self-reference via `@Autowired @Lazy private CatalogService self;` and called `self.remove()`. Calls to self invoke methods through the proxy, ensuring advice executes.

Q15: Would your aspect fire on a private method? On a static one? Justify your answer.

No. Spring AOP uses proxy-based interception. CGLIB creates a subclass and overrides public/protected methods; private methods cannot be overridden by subclasses, and static methods belong to the class rather than an instance. (Interception of private/static methods requires full AspectJ load-time or compile-time weaving, not Spring AOP proxies).

Q16: Name three Spring annotations that work through the same mechanism as your aspects.

- 1) `@Transactional` (Spring transaction management via `TransactionInterceptor`)
- 2) `@Cacheable` (Spring cache abstraction via `CacheInterceptor`)
- 3) `@Async` (Asynchronous method execution via `AsyncExecutionInterceptor`)